

## 1. Gaussian NB

```
In [64]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
In [65]: data = pd.read_csv('shopping_data.csv')
data.head()
```

```
Out[65]:
```

|   | CustomerID | Genre  | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Male   | 19  | 15                  | 39                     |
| 1 | 2          | Male   | 21  | 15                  | 81                     |
| 2 | 3          | Female | 20  | 16                  | 6                      |
| 3 | 4          | Female | 23  | 16                  | 77                     |
| 4 | 5          | Female | 31  | 17                  | 40                     |

```
In [66]: data.isnull().sum()
```

```
Out[66]: CustomerID      0
Genre      0
Age      0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64
```

```
In [67]: data_clean = data.drop(['CustomerID'], axis=1)
data_clean.head()
```

Out[67]:

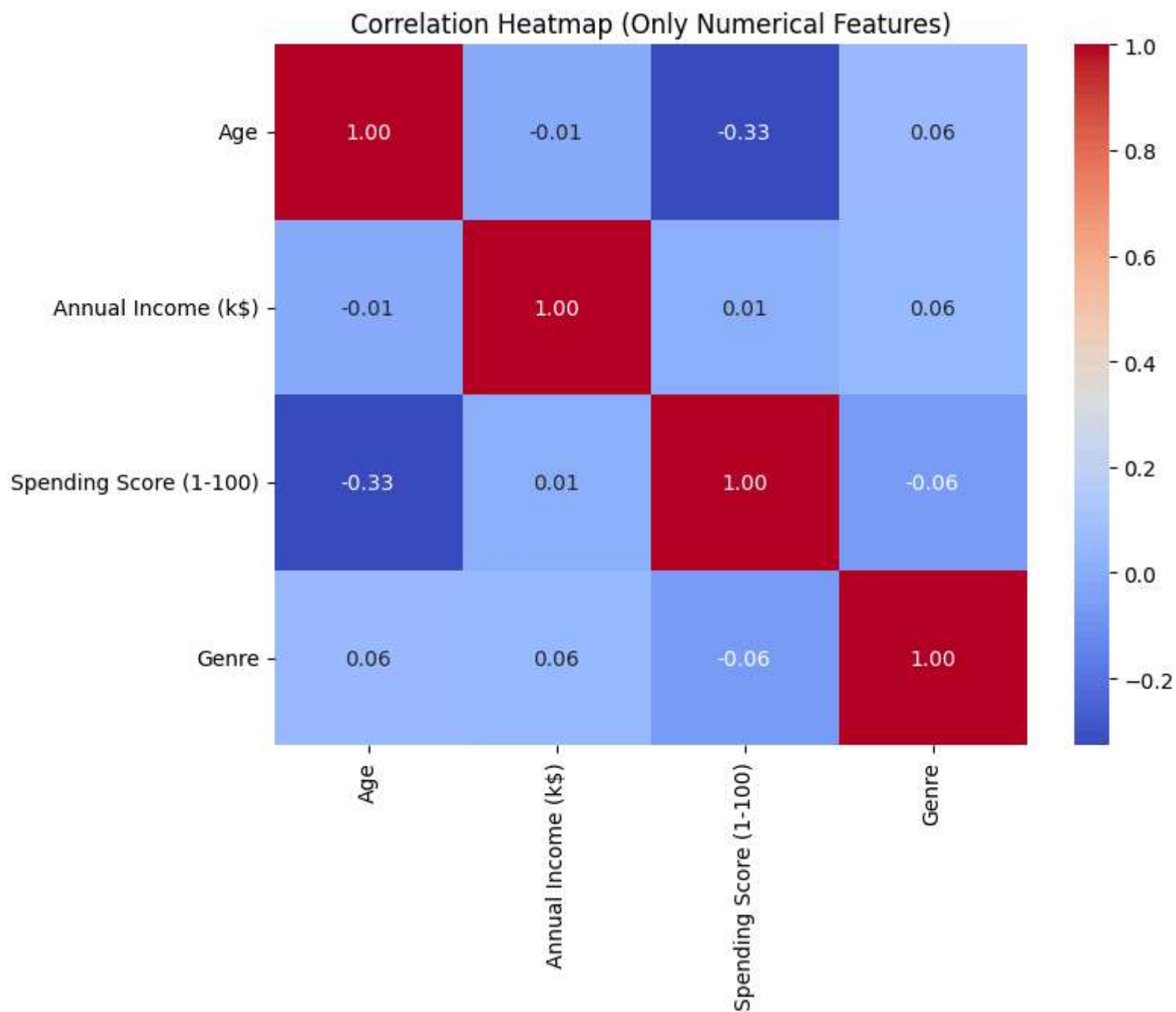
|   | Genre  | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|--------|-----|---------------------|------------------------|
| 0 | Male   | 19  | 15                  | 39                     |
| 1 | Male   | 21  | 15                  | 81                     |
| 2 | Female | 20  | 16                  | 6                      |
| 3 | Female | 23  | 16                  | 77                     |
| 4 | Female | 31  | 17                  | 40                     |

No missing values so we can proceed with label encoding correlation matrix

```
In [71]: data_clean['Genre'] = data_clean['Genre'].map({'Female': 0, 'Male': 1})
```

```
In [72]: num_cols = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)', 'Genre']

plt.figure(figsize=(8,6))
sns.heatmap(data_clean[num_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Only Numerical Features)")
plt.show()
```



```
In [73]: X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]  
y = data['Genre']
```

```
In [74]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [75]: gnb = GaussianNB()
gnb.fit(X_train_scaled, y_train)
```

```
Out[75]: GaussianNB ⓘ ?
GaussianNB()
```

```
In [76]: y_pred = gnb.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.6

Confusion Matrix:  
[[18 3]  
 [13 6]]

## 2. Bernoulli's NB

```
In [19]: import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import numpy as np
```

```
In [20]: breast_data = pd.read_csv('breastcancer.csv')
breast_data.head()
```

Out[20]:

|   | id       | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean | compactness_mean | concavity_mean | prior |
|---|----------|-----------|-------------|--------------|----------------|-----------|-----------------|------------------|----------------|-------|
| 0 | 842302   | M         | 17.99       | 10.38        | 122.80         | 1001.0    | 0.11840         | 0.27760          | 0.3001         |       |
| 1 | 842517   | M         | 20.57       | 17.77        | 132.90         | 1326.0    | 0.08474         | 0.07864          | 0.0869         |       |
| 2 | 84300903 | M         | 19.69       | 21.25        | 130.00         | 1203.0    | 0.10960         | 0.15990          | 0.1974         |       |
| 3 | 84348301 | M         | 11.42       | 20.38        | 77.58          | 386.1     | 0.14250         | 0.28390          | 0.2414         |       |
| 4 | 84358402 | M         | 20.29       | 14.34        | 135.10         | 1297.0    | 0.10030         | 0.13280          | 0.1980         |       |

5 rows × 33 columns



```
In [21]: breast_data.drop(columns=['id'], inplace=True)
breast_data['diagnosis'] = breast_data['diagnosis'].map({'M': 0, 'B': 1})
X = breast_data.drop(columns='diagnosis')
y = breast_data['diagnosis']
```

```
In [22]: # binarize the features
X_bin = X.apply(lambda col: (col > col.mean()).astype(int))
```

```
In [23]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_bin, y, test_size=0.2, random_state=42)
```

```
In [24]: from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score, classification_report

bnb = BernoulliNB()
bnb.fit(X_train, y_train)

y_pred = bnb.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```

Accuracy: 0.9824561403508771
              precision    recall  f1-score   support

     0       0.98        0.98        0.98        43
     1       0.99        0.99        0.99        71

 accuracy                   0.98        114
 macro avg       0.98        0.98        0.98        114
weighted avg       0.98        0.98        0.98        114

```

### 3. Multinational NB

```

In [8]: spam_data = pd.read_csv('spam.csv', encoding='latin1')
spam_data.head()

```

```

Out[8]:
   v1                                     v2  Unnamed: 2  Unnamed: 3  Unnamed: 4
0  ham  Go until jurong point, crazy.. Available only ...      NaN      NaN      NaN
1  ham                Ok lar... Joking wif u oni...      NaN      NaN      NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...      NaN      NaN      NaN
3  ham  U dun say so early hor... U c already then say...      NaN      NaN      NaN
4  ham  Nah I don't think he goes to usf, he lives aro...      NaN      NaN      NaN

```

```

In [11]: spam_data = spam_data.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'])
spam_data.head()

```

```

Out[11]:
   v1                                     v2
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...

```

```

In [12]: X = spam_data['v2']
y = spam_data['v1']

```

```
In [13]: from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
In [17]: from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer()  
X_train_vec = vectorizer.fit_transform(X_train)  
X_test_vec = vectorizer.transform(X_test)
```

```
In [18]: from sklearn.naive_bayes import MultinomialNB  
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
mnb = MultinomialNB()  
mnb.fit(X_train_vec, y_train)  
  
y_pred = mnb.predict(X_test_vec)  
  
print("Accuracy:", accuracy_score(y_test, y_pred))  
print(classification_report(y_test, y_pred))
```

Accuracy: 0.9820574162679426

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| ham          | 0.98      | 1.00   | 0.99     | 1453    |
| spam         | 0.97      | 0.89   | 0.93     | 219     |
| accuracy     |           |        | 0.98     | 1672    |
| macro avg    | 0.98      | 0.94   | 0.96     | 1672    |
| weighted avg | 0.98      | 0.98   | 0.98     | 1672    |